

Informe Tecnico-Explicativo:Codigo de la Vista de Apartamentos

Este manual explica la pantalla de apartamentos desde el codigo, no solo desde la interfaz.

Objetivo del documento:

1. Mostrar como esta estructurado el codigo.
2. Explicar para que sirve cada bloque.
3. Dar ejemplos reales para que una persona sin experiencia tecnica lo entienda.

Archivos analizados:

- src/components/apartments/Apartments.jsx
 - src/service/apartmentService.js
 - src/service/localStorage.js
-

1. Mapa general del funcionamiento

La vista sigue esta secuencia:

1. Verifica permisos de usuario.
 2. Carga apartamentos desde backend.
 3. Si backend falla, usa respaldo local.
 4. Permite buscar y filtrar.
 5. Permite crear, editar y cambiar estado.
 6. Refresca tabla y muestra mensajes de resultado.
-

2. Estructura del archivo principal Apartments.jsx

Fragmento real:

```
export default function Apartments() {
  const isLoggedIn = isUserLoggedIn()
  const isAdmin = isUserAdmin()
  const canAccess = isAdmin || isUserSupervisor()

  if (!isLoggedIn) {
    return <Navigate to="/login?redirect=/apartments" replace />
  }

  if (!canAccess) {
    return <Navigate to="/" replace />
  }
}
```

```
// ... estados, funciones y render  
}
```

Explicacion simple:

- La pantalla es una funcion llamada Apartments.
- Antes de mostrar cualquier dato, valida si el usuario puede entrar.
- Si no hay sesion, lo manda al login.
- Si hay sesion pero no tiene rol permitido, lo manda al inicio.

Ejemplo cotidiano:

- Es como una puerta con guardia:
 - sin credencial: no entras,
 - con credencial incorrecta: tampoco entras,
 - con credencial valida: pasas.

3. Importaciones: quien hace que

Fragmento real:

```
import {  
  isLoggedIn, isAdmin, isUserSupervisor,  
  getAllApartments, createApartmentLocal, updateApartmentLocal,  
  updateApartmentEstadoLocal  
} from '../service/localStorage'  
  
import {  
  getApartments,  
  createApartment,  
  updateApartment,  
  updateApartmentEstado  
} from '../service/apartmentService'
```

Lectura funcional:

- apartmentService: acceso a backend (servidor real).
- localStorage: plan B cuando backend falla.
- utils de alert/toast: mensajes y confirmaciones para el usuario.

4. Estados internos: memoria temporal de la pantalla

Fragmento real:

```
const [apartments, setApartments] = useState([])  
const [loading, setLoading] = useState(false)
```

```
const [searchTerm, setSearchTerm] = useState('')
const [showFilters, setShowFilters] = useState(false)
const [selectedFloor, setSelectedFloor] = useState('Todos')
const [selectedStatus, setSelectedStatus] = useState('Todos')
```

Explicacion:

- apartments: lista que se muestra en tabla.
- loading: indica si se esta cargando informacion.
- searchTerm: texto del buscador.
- showFilters: abre/cierra panel de filtros.
- selectedFloor y selectedStatus: valores elegidos en filtros.

Ejemplo:

- Si escribes A101, searchTerm guarda A101.
- Si eliges piso 2, selectedFloor guarda 2.

5. Carga inicial automatica

Fragmento real:

```
useEffect(() => {
  refreshAll()
}, [])
```

Que significa:

- Apenas la pantalla abre, ejecuta refreshAll una vez.
- Eso evita que el usuario tenga que presionar un boton de cargar.

6. Búsqueda inteligente (sin acentos y sin mayusculas)

Fragmento real:

```
const normalizeSearchText = (value = '') => {
  return String(value)
    .normalize('NFD')
    .replace(/[\u0300-\u036f]/g, '')
    .toLowerCase()
    .trim()
}
```

Para que sirve:

- Convierte el texto para comparar de forma mas flexible.

- Evita errores de coincidencia por acentos o mayúsculas.

Ejemplo:

- Usuario escribe Apartamento.
- Registro guardado: apartamento.
- Se considera coincidencia igual.

7. Índice de búsqueda por apartamento

Fragmento real:

```
const getApartmentSearchIndex = (apartment = {}) => {  
  const nombre = apartment.nombre || apartment.name || ''  
  const piso = apartment.piso ?? apartment.floor ?? ''  
  return normalizeSearchText(`${nombre} ${piso}`)  
}
```

Interpretación:

- Toma nombre y piso del apartamento.
- Los une en un solo texto para facilitar búsqueda.

Ejemplo:

- Apartamento: nombre A101, piso 3.
- Índice de búsqueda: a101 3.
- Si el usuario escribe 3, aparece.

8. Resolver estado en formato uniforme

Fragmento real:

```
const resolveApartmentStatus = (apartment = {}) => {  
  if (typeof apartment?.activo === 'boolean') {  
    return apartment.activo ? 'Activo' : 'Inactivo'  
  }  
  
  const backendStatus = apartment?.status || apartment?.estado  
  if (!backendStatus) return 'Activo'  
  
  const normalized = String(backendStatus).toLowerCase()  
  return normalized === 'activo' ? 'Activo' : 'Inactivo'  
}
```

Para no técnicos:

- Algunos datos llegan como activo true/false.
- Otros pueden llegar como texto status o estado.
- Esta funcion traduce todo a dos palabras estandar: Activo o Inactivo.

9. Filtro combinado (busqueda + piso + estado)

Fragmento real:

```
const filteredApartment = useMemo(() => {
  const term = normalizeSearchText(searchTerm)

  return apartments.filter((apartment) => {
    const matchesSearch = !term ||
getApartmentSearchIndex(apartment).includes(term)
    const matchesFloor = selectedFloor === 'Todos' ||
String(apartment?.piso ?? '') === selectedFloor
    const matchesStatus = selectedStatus === 'Todos' ||
resolveApartmentStatus(apartment) === selectedStatus
    return matchesSearch && matchesFloor && matchesStatus
  })
}, [apartments, searchTerm, selectedFloor, selectedStatus])
```

Que hace exactamente:

- Solo muestra filas que cumplen los tres criterios simultaneamente.
- Si un filtro esta en Todos, ese filtro se ignora.

Ejemplo concreto:

- Busqueda: A1
- Piso: 2
- Estado: Activo

Solo veras apartamentos que contengan A1, esten en piso 2 y esten activos.

10. Carga de datos con estrategia hibrida

Fragmento real:

```
const refreshAll = async () => {
  setLoading(true)
  try {
    const apartmentsFromApi = await getApartments()
    setApartments(Array.isArray(apartmentsFromApi) ? apartmentsFromApi
: [])
  } catch (err) {
    setApartments(getAllApartments())
  } finally {
```

```
        setLoading(false)
    }
}
```

Lectura en lenguaje simple:

1. Marca la pantalla como cargando.
2. Intenta traer datos del servidor.
3. Si falla, usa copia local del navegador.
4. Quita estado de cargando.

11. Crear apartamento: flujo completo de código

Fragmento real resumido:

```
const result = await Swal.fire({
  title: 'Crear apartamento',
  preConfirm: () => {
    const nombre = document.getElementById('swal-nombre')?.value?.trim() || ''
    const piso = Number(document.getElementById('swal-piso')?.value)
    const activo = document.getElementById('swal-activo')?.value === 'true'

    if (!nombre) {
      Swal.showValidationMessage('El nombre es obligatorio')
      return false
    }
    if (!Number.isInteger(piso)) {
      Swal.showValidationMessage('El piso debe ser un numero entero')
      return false
    }

    return { nombre, piso, activo }
  }
})

try {
  await createApartment(result.value)
} catch (err) {
  const localResult = createApartmentLocal(result.value)
  if (!localResult?.success) throw new Error(localResult?.error)
}
```

Explicación paso a paso:

1. Abre una ventana modal para pedir datos.
2. Valida nombre y piso antes de enviar.
3. Intenta crear en backend.

4. Si backend falla, crea en localStorage.
5. Si ambos fallan, muestra error.

12. Editar apartamento: diferencia con crear

Fragmento real resumido:

```
const apartmentId = apartment.id

try {
  await updateApartment(apartmentId, result.value)
} catch (err) {
  const localResult = updateApartmentLocal(apartmentId, result.value)
  if (!localResult?.success) throw new Error(localResult?.error)
}
```

Concepto clave:

- Crear no tiene id previo.
- Editar necesita id para saber que registro modificar.

Ejemplo:

- Editar apartamento id 12 cambia solo ese registro.

13. Cambio de estado (activar/desactivar)

Fragmento real resumido:

```
confirmAction({ title, text, confirmText }).then(async (ok) => {
  if (!ok) return

  try {
    await updateApartmentEstado(id, nextActivo)
  } catch (err) {
    if (err?.response?.status === 403) {
      showErrorToast('Permiso denegado (403)')
      return
    }
    const localResult = updateApartmentEstadoLocal(id, nextActivo)
    if (!localResult?.success) throw new Error(localResult?.error)
  }

  await refreshAll()
})
```

Interpretacion:

- Primero pide confirmacion humana.
- Luego intenta backend.
- Si no hay permiso (403), no aplica fallback y corta.
- Si es otro error, usa fallback local.

14. Servicio de backend apartmentService.js

Fragmento real:

```
const BASE = '/api/v1/apartments';

export const getApartments = () => api.get(BASE).then(r => r.data);
export const getApartment = (id) => api.get(`${BASE}/${id}`).then(r => r.data);
export const createApartment = (payload) => api.post(BASE, payload).then(r => r.data);
export const updateApartment = (id, payload) => api.put(`${BASE}/${id}`, payload).then(r => r.data);
export const updateApartmentEstado = (id, activo) => api.patch(`${BASE}/${id}/estado`, null, { params: { activo } }).then(r => r.data);
```

En lenguaje de negocio:

- getApartments: listar todos.
- getApartment: traer uno por id.
- createApartment: crear uno nuevo.
- updateApartment: actualizar uno existente.
- updateApartmentEstado: activar o desactivar.

15. Respaldo local en localStorage.js

15.1 Claves de almacenamiento

Fragmento real:

```
const APARTMENTS_KEY = 'apartments';
const NEXT_APARTMENT_ID_KEY = 'nextApartmentId';
```

Que representan:

- APARTMENTS_KEY: donde se guarda la lista.
- NEXT_APARTMENT_ID_KEY: contador para crear ids locales.

15.2 Normalizacion de datos

Fragmento real:

```
const normalizeApartmentDto = (dto = {}) => ({
  id: dto?.id !== null ? Number(dto.id) : null,
  nombre: dto?.nombre || '',
  piso: dto?.piso !== null && dto?.piso !== '' ? Number(dto.piso) : null,
  activo: typeof dto?.activo === 'boolean' ? dto.activo : true
});
```

Objetivo:

- Convertir datos de entrada a un formato consistente.
- Evitar errores por tipos de datos diferentes.

15.3 Validaciones de negocio local

Fragmento real:

```
if (!normalized.nombre.trim()) {
  return { success: false, message: 'Error al crear apartamento', error:
    'El nombre es obligatorio' };
}
if (!Number.isInteger(normalized.piso)) {
  return { success: false, message: 'Error al crear apartamento', error:
    'El piso es obligatorio y debe ser numerico' };
}

const existsName = apartments.some(
  a => (a.nombre || '').trim().toLowerCase() ===
    normalized.nombre.trim().toLowerCase()
);
```

Que valida:

- nombre obligatorio,
- piso numerico,
- nombre unico (sin duplicados).

16. Estructura visual de la tabla y acciones

Fragmento real:

```
<CTableHead color="light">
  <CTableRow>
    <CTableHeaderCell>Apartamento</CTableHeaderCell>
    <CTableHeaderCell>Piso</CTableHeaderCell>
    <CTableHeaderCell>Estado</CTableHeaderCell>
```

```
<CTableHeaderCell>Acciones</CTableHeaderCell>
</CTableRow>
</CTableHead>
```

Lectura funcional:

- Cada fila representa un apartamento.
- Acciones disponibles por fila:
 - editar,
 - activar,
 - desactivar.

17. Escenarios explicados con flujo de código

Escenario A: backend operativo

1. refreshAll usa getApartments.
2. apartments se llena con respuesta del servidor.
3. Tabla muestra datos oficiales.

Escenario B: backend caído

1. refreshAll falla en try.
2. catch ejecuta getAllApartments.
3. Tabla se mantiene funcional con datos locales.

Escenario C: usuario sin permisos

1. canAccess da false.
2. Componente retorna Navigate al inicio.
3. No llega a cargar datos ni renderizar tabla.

18. Riesgos y observaciones actuales del código

1. Hay dos botones de editar en cada fila y ambos hacen lo mismo.
2. La paginación es visual, aun no pagina realmente.
3. Si se trabaja en fallback local por mucho tiempo, puede existir diferencia temporal con el backend.

19. Glosario técnico para no técnicos

- Estado de React: memoria temporal de una pantalla.
 - Hook useEffect: acción automática al abrir pantalla.
 - Hook useMemo: calcula resultados y evita trabajo repetido.
 - DTO: estructura de datos que viaja entre frontend y backend.
 - Fallback: alternativa cuando falla el camino principal.
-

20. Conclusion del informe

El código de la vista de apartamentos está construido con un enfoque robusto:

1. Controla acceso antes de cualquier operación.
2. Mantiene experiencia continua con backend + fallback local.
3. Aplica validaciones en creación y edición.
4. Estandariza estados para renderizar de forma uniforme.
5. Se mantiene alineado con la estructura visual de la vista de usuarios.

Este documento permite entender no solo que ve el usuario, sino como el código lo consigue paso a paso.